Explore | Expand | Enrich

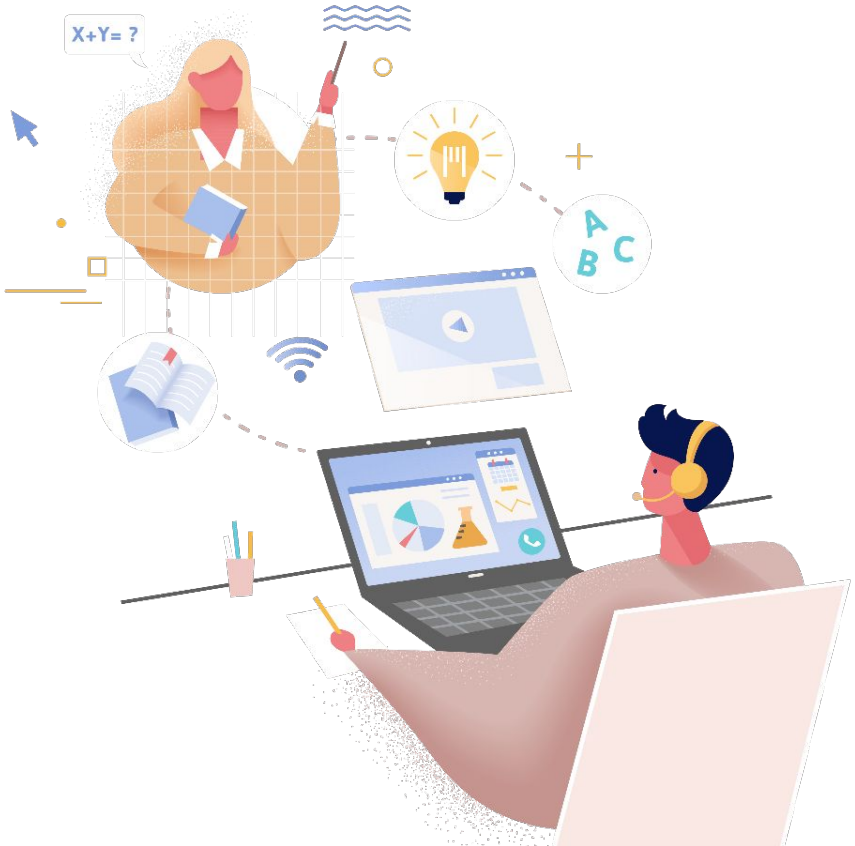
<Codemithra />™

<Codemithra />™

Basic Calculator

TOPICS

- Introduction
- Algorithm Pseudocode
- Example
- Code
- Time and space complexity
- Advantages and disadvantages
- Interview Questions



Introduction

- A Basic Calculator evaluates a mathematical expression given as a string.
- The expression contains integers, '+', '-', '(', ')', and spaces.
- Goal: Return the result after evaluating the expression while handling operator precedence and parentheses correctly.
- Example:
  - Input: "1 + 1" → Output: 2
  - Input: "2-(1+2)" → Output: -1

Explanation

- Parentheses alter normal left-to-right evaluation order.
- Stack is used to:
  - Remember intermediate results.
  - Handle nested parentheses correctly.
- Operations supported: **addition (+)** and **subtraction (-)** only.

Algorithm (Stack-based Approach)

1. Initialize:
   - result = 0, sign = 1 (1 for +, -1 for -), and a stack.
2. Traverse each character in the expression:
3.
   - If digit → parse full number and add to result with correct sign.
   - If + or - → update sign.
   - If (→ push current result and sign onto stack, reset result = 0, sign = 1.
   - If) → pop sign and previous result from stack, compute result.
4. Return the final result.

Algorithm Pseudocode

```
function calculate(expression):
   result = 0
   sign = 1
   stack = []
   i = 0

   while i < length(expression):
      if expression[i] is digit:
         num = 0
         while i < length(expression) and
expression[i] is digit:
            num = num * 10 + (expression[i] - '0')
            i++
         result += sign * num
         continue
      else if expression[i] == '+':
         sign = 1
      else if expression[i] == '-':
         sign = -1
      else if expression[i] == '(':
         push(result)
```

```
push(sign)
         result = 0
         sign = 1
      else if expression[i] == ')':
         prevSign = pop()
         prevResult = pop()
         result = prevResult + prevSign * result
      i++

   return result
```

Example

Input 1:

s = "1 + 1"

Output: 2

Input 2:

s = "2-(1+2)"

Output: -1

Code

```
public class Main {

   public static int calculate(String s) {
      java.util.Stack<Integer> stack = new
java.util.Stack<>();
      int result = 0;
      int sign = 1;
      int num = 0;
for (int i = 0; i < s.length(); i++) {
         char ch = s.charAt(i);

         if (Character.isDigit(ch)) {
            num = num * 10 + (ch - '0');
         } else if (ch == '+') {
            result += sign * num;
            num = 0;
            sign = 1;
         } else if (ch == '-') {
            result += sign * num;
            num = 0;
            sign = -1;
         }
```

```
else if (ch == '(') {
            stack.push(result);
            stack.push(sign);
            result = 0;
            sign = 1;
         } else if (ch == ')') {
            result += sign * num;
            num = 0;
            result *= stack.pop();
            result += stack.pop();   }
      }
      result += sign * num;
      return result;
   }
```

```
public static void main(String[] args) {
    String input1 = "1 + 1";
    String input2 = "2-(1+2)";

    System.out.println("Input: " + input1);
    System.out.println("Output: " +
calculate(input1));

    System.out.println("Input: " + input2);
    System.out.println("Output: " +
calculate(input2));
  }
}
```

Time Complexity: O(n)

1. **Single Scan:**
   - The algorithm processes each character in the input string exactly once (digits, operators, or parentheses).
   - No nested loops, so the total operations are proportional to the length of the string → O(n).
2. **Constant-time operations:**
   - Each step (pushing/popping from stack, updating result, parsing a number) takes constant time.
   - Therefore, even with nested parentheses, the time remains linear.

Space Complexity: O(n)

1. **Stack Usage:**
   - A stack is required to store intermediate results and signs when encountering '('.
   - In the worst case (deeply nested parentheses), all characters may be pushed → **O(n)**.

2. **Minimal extra storage:**

- Aside from the stack, only a few variables (result, sign, num) are used.
- Thus, auxiliary space remains **linear in the worst case**.

Advantages

1. Handles **nested parentheses** correctly.
2. Works in **linear time**, efficient for large input strings.
3. Can be extended to support more operators.

Disadvantages

1. Extra **stack space** required (O(n)).
2. Only supports + and - (limited functionality).
3. Needs careful handling of **multi-digit numbers** and spaces.

Interview Questions:

QUESTION 1

Why do we need a stack in this problem?

Answer:

- To remember previous results and signs when entering/exiting parentheses.
- Without stack, nested parentheses cannot be handled properly.

QUESTION 2

How would you extend this calculator to handle * and /?

Answer:

1. Use operator precedence rules and evaluate multiplication/division before addition/subtraction.
2. Could implement using two stacks (operators and operands).

QUESTION 3

What is the difference between evaluating from left-to-right vs operator precedence?

Answer:

- Left-to-right works only for equal-precedence operators.
- With precedence, * and / must be evaluated before + and -.

ETHNUS™
Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com